

Interview Readiness Guide

The "meta" volume — perform, don't just know

Target level: Mid-level software-engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

Why this guide exists

Every other volume in your library builds **knowledge** — patterns, problems, designs, Java internals. This one builds **performance**. At this stage the gap between you and an offer is rarely "I didn't know the algorithm." It's one of these:

- You *knew* the pattern but took 8 minutes to see it, and ran out of time.
- You jumped to code, hit a wall at minute 30, and couldn't recover gracefully.
- You solved it silently, so the interviewer couldn't follow your thinking.
- You forgot to test, shipped a bug, and lost the "attention to detail" signal.
- You froze on a variation of a problem you'd solved before.

None of those are knowledge gaps. They're **execution** gaps — and they're exactly what this guide drills.

What's inside

1. **The pattern-recognition decision tree** — turn the clues in a problem statement into a pattern in under 60 seconds. This is the skill that separates "I eventually got it" from "I saw it immediately."
2. **The complexity cheat-sheet** — the numbers to state before you're asked, plus the "what does the target complexity tell me about the intended approach" trick.
3. **The 45-minute playbook** — a minute-by-minute script for the coding round, so you never run out of time or freeze on where to start.
4. **The common-mistakes checklist** — the silent-rejection traps, and how to avoid each.
5. **Four fully-worked mock transcripts** — what a strong candidate *actually says out loud*, start to finish, including a curveball and a bug they catch. This is the part most people never practice.

How to use it

- Read the decision tree and playbook **once**, then keep them next to you during practice.
 - After every practice problem, run the **common-mistakes checklist** against your own solve.
 - Read each **mock transcript out loud** — literally speak the candidate's lines. The goal is to make "thinking out loud" feel natural, because under stress you default to silence.
 - Two weeks before a loop, do **timed 45-minute mocks** using the playbook. Knowing 200 problems helps far less than doing 10 problems perfectly *under the clock, out loud*.
-

1. The Pattern-Recognition Decision Tree

The one skill this trains. In the first 60 seconds, strip the story down to three things — **input, output, constraint** — then let the clues route you to a pattern. Strong candidates don't "remember" the answer; they *recognize* the shape. This section is the routing table.

Step 0: De-story the problem (always do this first)

Before anything, answer three questions out loud:

1. INPUT — what am I given? (array? string? tree? graph? stream? matrix?)
2. OUTPUT — what must I return? (a number? a boolean? all solutions? the best one?)
3. CONSTRAINT— what's the size (n)? sorted? distinct? negative? cyclic?

The **constraint** is the biggest tell — the size of `n` almost always leaks the intended complexity (see §2). "Return **all** X" screams backtracking; "return the **best/most/min** X" screams DP or greedy; "have I **seen** this" screams hashing.

The master decision tree

START: what is the INPUT?

- ARRAY / STRING
 - Is it SORTED (or can I sort it)?
 - find a pair/triplet summing to target TWO POINTERS
 - find a position / boundary / "first true" BINARY SEARCH
 - answer is monotonic in some guess BINARY SEARCH ON ANSWER
 - Contiguous subarray/substring, "longest/shortest/at most K"
 - SLIDING WINDOW
 - Many range-sum / subarray-sum-equals-K queries
 - PREFIX SUM (+ hash map)
 - Many range UPDATES DIFFERENCE ARRAY
 - "Next greater / smaller", spans, histogram
 - MONOTONIC STACK
 - Max/min of every window of size k MONOTONIC DEQUE
 - "Have I seen it / count / group / dedupe" HASHING / FREQUENCY MAP
 - "Top K / Kth largest / streaming merge" .. HEAP
- LINKED LIST
 - cycle / middle / nth-from-end FAST & SLOW POINTERS
 - reverse / reorder / merge POINTER MANIPULATION
- TREE
 - level-by-level / shortest depth BFS (queue)
 - path / validate / ancestor / diameter ... DFS (recursion)
 - "kth smallest in BST / sorted order" IN-ORDER DFS
- GRAPH (grid counts as a graph!)
 - shortest steps, unweighted BFS
 - shortest steps from MANY starts MULTI-SOURCE BFS
 - shortest path, weights 0/1 0-1 BFS (deque)
 - shortest path, positive weights DIJKSTRA
 - shortest path, negative / $\leq K$ stops BELLMAN-FORD
 - connectivity / components / "merge" UNION-FIND or DFS
 - ordering with dependencies / "is there a cycle"
 - TOPOLOGICAL SORT
 - "explore all of a region / count islands" GRID DFS/BFS (flood fill)
 - "longest path in a DAG" MEMOIZED DFS (DP on graph)
- WHAT IS THE OUTPUT? (when input alone doesn't decide it)
 - "ALL subsets / permutations / combinations / arrangements"
 - BACKTRACKING
 - "count the ways" or "best/max/min value" with overlapping choices
 - DYNAMIC PROGRAMMING
 - "best value, one greedy choice at each step is provably safe"
 - GREEDY
 - "overlapping ranges / scheduling / meeting rooms"
 - INTERVALS (sort + sweep)

The clue → pattern lookup table

Keep this next to you while practicing. When you hear the phrase, reach for the pattern.

Phrase in the problem	Pattern to try first
"sorted array", "find a pair/triplet"	Two pointers
"longest/shortest substring/subarray with..."	Sliding window
"at most K distinct / K replacements"	Sliding window
"subarray sum equals / divisible by K"	Prefix sum + hash map
"range update, many times"	Difference array
"how many times have I seen...", "group by..."	Hashing
"sorted, find position / min / boundary"	Binary search
"minimize the maximum" / "can we do it in X?"	Binary search on the answer
"next greater", "warmer day", "span", "histogram"	Monotonic stack
"max of every window"	Monotonic deque
"top K", "Kth largest", "median of a stream"	Heap (one or two)
"cycle in a linked list", "middle node"	Fast & slow pointers
"level order", "shortest steps", "min moves"	BFS
"all paths", "does a path exist", "flood the region"	DFS
"course prerequisites", "build order", "cycle in deps"	Topological sort
"islands", "provinces", "accounts merge", "connected"	Union-Find / DFS
"all subsets / permutations / combinations"	Backtracking
"count the ways", "min cost", "max profit"	DP
"intervals", "meeting rooms", "merge ranges"	Intervals (sort first)
"generate the Nth / all valid strings"	Backtracking or DP

Two traps in recognition

Trap 1 — the disguise. Companies wrap patterns in their domain ("cheapest N consecutive hotel nights" = sliding window; "flight route with $\leq K$ layovers" = Bellman-Ford). Always de-story first (§Step 0). If the story confuses you, replace nouns with `array`, `node`, `cost`.

Trap 2 — two patterns at once. Hard problems often compose: *Maximal Rectangle* = DP row heights + monotonic stack; *Longest Increasing Path* = graph + memoization. If one pattern gets you 80% there but stalls, ask "what pattern solves the leftover subproblem?"

What to say out loud during recognition

"Let me restate the input and output... I'm given [...], I need to return [...], and `n` can be up to [...]. That upper bound rules out $O(n^2)$, so I'm looking for $O(n \log n)$ or better. The

phrase 'longest substring with at most K distinct' points me at a **sliding window**. Let me sketch that before I code."

That single paragraph signals: you de-story, you use constraints, you name the pattern, and you plan before coding. It's the strongest opening you can give.

2. The Complexity Cheat-Sheet

The one skill this trains. Stating complexity *before you're asked* signals seniority. And reading the constraint *backwards* — from the size of `n` to the intended complexity — often tells you the pattern before you've fully understood the problem. This section is both the reference and the trick.

The trick: `n` leaks the intended complexity

Interviewers pick `n` deliberately. The upper bound on the input size is a **hint** at the solution they want. Memorize this table — it's the highest-leverage thing in this guide.

If <code>n</code> is up to...	The intended solution is probably...	Because roughly...
$\leq 10-12$	Backtracking / permutations / bitmask DP	$O(n!)$ or $O(2^n)$ is fine
$\leq 20-25$	Bitmask DP, meet-in-the-middle	$O(2^n) \approx 1\text{M}$
$\leq 100-500$	$O(n^3)$ DP (interval / Floyd-Warshall)	$n^3 \approx 10^7$
$\leq 1,000-5,000$	$O(n^2)$ — 2D DP, nested loops	$n^2 \approx 10^7$
$\leq 10^5-10^6$	$O(n \log n)$ — sort, heap, binary search	fits $\sim 10^8$ ops/sec
$\leq 10^7-10^8$	$O(n)$ or $O(n \log \log n)$ — single pass, sieve	must be linear-ish
$> 10^8$ or "huge" / streaming	$O(\log n)$ or $O(1)$ — math, binary search, formula	can't even read it all

How to use it in the room: "*n is up to 10^5 , so an $O(n^2)$ approach ($\sim 10^{10}$ ops) is too slow — I need $O(n \log n)$ or $O(n)$. That pushes me toward sorting or a hash map rather than nested loops.*" You've just narrowed the solution space before writing a line.

Complexity of the operations you'll actually use (Java)

Structure / operation	Time
<code>ArrayList</code> get / set / add-at-end	$O(1)$ amortized
<code>ArrayList</code> add/remove in middle	$O(n)$
<code>HashMap</code> / <code>HashSet</code> get / put / contains	$O(1)$ average
<code>TreeMap</code> / <code>TreeSet</code> get / put / ceiling / floor	$O(\log n)$
<code>PriorityQueue</code> add / poll	$O(\log n)$
<code>PriorityQueue</code> peek	$O(1)$
<code>ArrayDeque</code> push / pop / offer / poll (both ends)	$O(1)$
<code>Arrays.sort</code> / <code>Collections.sort</code>	$O(n \log n)$

Structure / operation	Time
String concatenation in a loop (+)	$O(n^2)$ — use <code>StringBuilder</code> !
<code>StringBuilder.append</code>	$O(1)$ amortized
<code>substring</code>	$O(n)$ (copies)

The complexity of the patterns in your library

Pattern	Typical time	Typical space
Two pointers	$O(n)$	$O(1)$
Sliding window	$O(n)$	$O(1)$ or $O(k)$
Prefix sum (+ hash)	$O(n)$	$O(n)$
Binary search	$O(\log n)$	$O(1)$
Binary search on answer	$O(n \log(\text{range}))$	$O(1)$
Hashing / frequency	$O(n)$	$O(n)$
Heap "top K"	$O(n \log k)$	$O(k)$
BFS / DFS (graph)	$O(V + E)$	$O(V)$
Dijkstra (binary heap)	$O(E \log V)$	$O(V)$
0-1 BFS	$O(V + E)$	$O(V)$
Union-Find (with compression)	$O(E \cdot \alpha) \approx O(E)$	$O(V)$
Topological sort	$O(V + E)$	$O(V)$
Backtracking (subsets)	$O(2^n)$	$O(n)$ recursion
Backtracking (permutations)	$O(n!)$	$O(n)$ recursion
1D DP	$O(n)$	$O(n) \rightarrow$ often $O(1)$
2D / interval DP	$O(n^2)$	$O(n^2) \rightarrow$ often $O(n)$
Monotonic stack / deque	$O(n)$	$O(n)$

Space you forget to count

- **Recursion stack** is space. A DFS `n` deep is $O(n)$ space even with no explicit structure.
- **The output** is sometimes excluded ("ignoring output") and sometimes not — say which you mean. For "return all subsets", the output itself is $O(2^n)$.
- **Hash maps of prefixes** are $O(n)$ even though the code looks like $O(1)$ per step.

Amortized vs. worst case — say the right word

- `HashMap.get` is **$O(1)$ average**, $O(n)$ worst case (pathological collisions). Say "average".
 - `ArrayList.add` is **$O(1)$ amortized** (occasional resize doubles capacity). Say "amortized".
 - Monotonic stack is $O(n)$ **because each element is pushed and popped at most once** — say *why* it's linear even though there's a `while` inside a `for`. That explanation is a strong signal.
-

What to say out loud

"This is $O(n \log n)$ time — the sort dominates — and $O(n)$ space for the hash map. The recursion adds $O(h)$ stack space where h is the tree height, so $O(\log n)$ balanced, $O(n)$ worst case. I could drop the space to $O(1)$ if I sort in place, at the cost of mutating the input — want me to do that?"

Offering the space/time tradeoff *and its cost* is exactly the senior signal these loops reward.

3. The 45-Minute Playbook

The one skill this trains. A coding round is not "solve the problem." It's "demonstrate a strong problem-solving *process* inside a fixed clock." Candidates fail not because they can't solve it, but because they mismanage the 45 minutes — 20 minutes silent, then a panic. This is a minute-by-minute script. Internalize it so you never wonder "what do I do now?"

The timeline at a glance

0:00 — 0:05	UNDERSTAND	restate, ask questions, write examples
0:05 — 0:12	PLAN	brute force out loud → optimize → pick approach → get buy-in
0:12 — 0:32	CODE	narrate as you write; clean, compilable Java
0:32 — 0:40	TEST	dry-run the example by hand; fix bugs you find
0:40 — 0:45	ANALYZE	state complexity; discuss tradeoffs & follow-ups

Adjust for a 60-minute round by stretching CODE and adding a second problem or a follow-up extension. The **ratios** matter more than the exact minutes: ~15% understand, ~15% plan, ~45% code, ~20% test, ~5% wrap.

Phase 1 — UNDERSTAND (0:00–0:05)

Do: - **Restate the problem** in your own words. ("So I'm given..., and I need to return...") - **Ask clarifying questions** — they're free signal: - Input size / range of values? (drives complexity — see §2) - Can the input be empty / null / a single element? - Are values sorted? distinct? can they be negative? - Are there duplicates in the output? What order? - Can I mutate the input? - **Write 1–2 concrete examples** yourself, including one **edge case** (empty, one element, all-same). This catches misunderstandings *before* they cost you code.

Don't: start coding. Don't assume — every assumption you voice is a chance for the interviewer to correct you cheaply.

Red flag you're rushing: you've written zero examples and you're reaching for the keyboard.

Phase 2 — PLAN (0:05–0:12)

Do: - **State the brute force first**, out loud, with its complexity. ("The obvious way is to check every pair — $O(n^2)$. Let me see if I can do better.") This buys goodwill and often reveals the optimization. - **Name the pattern** (use the §1 decision tree). ("This looks like a sliding window because...") - **Sketch the approach** — the diagram, the data structures, the key invariant — *before* code. - **Get buy-in:** "My plan is a hash map keyed by prefix sum, one pass, $O(n)$. Sound good?" Interviewers will often nudge you if you're heading wrong — that nudge is worth minutes.

Don't: silently decide. If you code without stating the plan and it's wrong, you've wasted 20 minutes the interviewer could have saved with one sentence.

The single most important habit: *plan out loud and get a nod before you type.*

Phase 3 — CODE (0:12–0:32)

Do: - **Narrate as you write.** ("I'll initialize the map with `{0:1}` for the empty prefix...") - Write **clean, compilable Java** — real method signatures, meaningful names, no pseudo-code. - **Handle the structure first**, details second. Stub a helper and fill it in. - If you get stuck, **say what you're stuck on** — thinking out loud lets the interviewer help. - Keep the earlier examples visible; code *toward* them.

Don't: - Go silent for more than ~20 seconds. Silence reads as "lost." - Prematurely optimize micro-stuff. Get it correct, then tighten. - Rewrite from scratch at minute 30. Patch forward.

If you're stuck at minute 25: fall back to the brute force and get *something* working. A correct $O(n^2)$ beats an unfinished $O(n)$. Say: "Let me get the brute force down so we have a working solution, then optimize if time allows." That's a recovery, not a defeat.

Phase 4 — TEST (0:32–0:40)

Do: - **Dry-run your own example by hand**, tracing variables line by line. Narrate the values. - **Test the edge cases** you wrote in Phase 1: empty, single element, all-same, negatives, the max size (does it overflow? use `long`?). - **Fix bugs calmly.** Finding your own bug is a *positive* signal — it shows rigor. Say "let me trace this — ah, off-by-one here, the loop should be `<=`."

Don't: declare victory the moment the code is written. "It compiles" $\neq$ "it's correct". Skipping the dry-run is one of the top silent-rejection causes (see §4).

The overflow check: for anything summing or multiplying ints, ask "can this exceed `Integer.MAX_VALUE`?" If yes, use `long`. Interviewers love this catch.

Phase 5 — ANALYZE & WRAP (0:40–0:45)

Do: - **State final complexity** (time and space), and *why*. - **Offer a tradeoff:** "I used $O(n)$ extra space for the map; I could do $O(1)$ space by sorting first, but that's $O(n \log n)$ time and mutates the input." -

Volunteer follow-ups you'd handle with more time: "If the input were streaming, I'd...", "To scale this across a cluster, I'd...". This shows range.

Don't: end abruptly with "done." End with the complexity and one thoughtful extension.

The recovery protocol (when it goes wrong)

Everyone gets stuck. What's assessed is *how you recover*:

STUCK on recognition? -> re-read; de-story to (input, output, constraint); try the brute force
STUCK on the optimization? -> ship the brute force, then optimize the bottleneck
STUCK on a bug? -> dry-run with the smallest failing example, narrate each variable
STUCK completely? -> say what you know, what you've ruled out, and ask a specific question

The worst move is **silent flailing**. The best move is **narrated struggle** — it turns a dead end into a collaboration, and interviewers grade collaboration.

One-line summary to memorize

Understand → **plan out loud** → **get buy-in** → **code while narrating** → **test by hand** → **state complexity**. Never silent, never straight to code, always test before "done."

4. The Common-Mistakes Checklist

The one skill this trains. Most rejections aren't "couldn't solve it" — they're accumulated small negatives that add up to "not quite a hire." These are the silent rejection traps. Run this checklist against every practice solve until avoiding them is automatic.

Category A — Process mistakes (the silent killers)

Mistake	Why it costs you	The fix
Jumping straight to code	Signals you don't plan; you code the wrong thing	Always do §3 Phase 1–2 first
Coding in silence	Interviewer can't follow or help; reads as "lost"	Narrate every step
Not asking clarifying questions	Miss constraints; solve the wrong problem	Ask 3–5 questions up front
Not writing examples	Misunderstand the problem; no test data later	Write 2 examples + 1 edge case
Not stating the brute force	Miss the goodwill and the stepping stone	Say it out loud, then optimize
Skipping the dry-run	Ship bugs; lose the rigor signal	Trace your example by hand before "done"
Ending with just "done"	Waste the seniority signal	State complexity + one tradeoff

Category B — Correctness bugs (the ones a dry-run catches)

Bug	Where it hides	Guard
Off-by-one	loop bounds <code><</code> vs <code><=</code> , <code>n-1</code> , window edges	trace the first & last iteration by hand
Integer overflow	<code>left + right</code> in binary search, sums, products	use <code>left + (right-left)/2</code> ; use <code>long</code>
Empty / null input	<code>arr[0]</code> , <code>s.charAt(0)</code> , <code>list.get(0)</code>	check <code>if (n == 0) return ... first</code>
Single-element input	two-pointer / window logic	test <code>n == 1</code> explicitly
Duplicate handling	3Sum, subsets, permutations	sort + skip equal neighbors
Modifying a collection while iterating	<code>ConcurrentModificationException</code>	iterate a copy or use an iterator's <code>remove</code>

Bug	Where it hides	Guard
<code>==</code> vs <code>.equals()</code> on objects	<code>Integer</code> , <code>String</code> comparisons	use <code>.equals()</code> ; <code>==</code> only for primitives
Uninitialized / wrong default	max starting at 0 when values can be negative	start at <code>Integer.MIN_VALUE</code>
Visited-set placement (BFS)	mark visited on <i>enqueue</i> , not <i>dequeue</i>	else nodes enter the queue many times
Recursion base case	missing or wrong → stack overflow / wrong answer	write the base case first

Category C — Java-specific traps

Trap	Detail
<code>String</code> + in a loop	$O(n^2)$ — build with <code>StringBuilder</code> , <code>append</code> , <code>one.toString()</code>
<code>int</code> overflow silent	<code>int a * b</code> overflows with no error — cast to <code>long</code> before multiplying
<code>Arrays.asList</code> on primitives	<code>Arrays.asList(intArray)</code> makes a <code>List<int[]></code> of size 1 — box first
<code>PriorityQueue</code> is a min-heap	for max-heap pass <code>Collections.reverseOrder()</code> or a comparator
<code>HashMap</code> iteration order	not insertion order — use <code>LinkedHashMap</code> if order matters
Integer caching <code>-128..127</code>	<code>Integer a = 1000; a == 1000</code> may be false — always <code>.equals()</code>
<code>%</code> of a negative	<code>-7 % 3 == -1</code> in Java — normalize with <code>((x % k) + k) % k</code>
<code>char</code> arithmetic	<code>s.charAt(i) - 'a'</code> gives an index; remember it's an <code>int</code>
Removing from <code>ArrayList</code> by index vs value	<code>remove(int)</code> = index, <code>remove(Object)</code> = value — ambiguous for <code>Integer</code>

Category D — Communication mistakes

Mistake	Better move
Arguing when the interviewer hints	Treat hints as gifts: "Good point, let me reconsider..."
Pretending to know something you don't	"I'm not certain of the exact API — I'd look it up; the idea is..."
Over-explaining trivial code	Narrate <i>decisions</i> , not every <code>i++</code>
Going quiet when stuck	Narrate the struggle: "I'm stuck because X; my options are Y or Z"

Mistake	Better move
Not managing the clock	Glance at time; at minute 25 with no working code, fall back to brute force
Dismissing edge cases	"Let me make sure empty input and a single element both work"

The 30-second pre-submit checklist

Before you say "I think this is correct," silently run:

```
[ ] Did I handle empty / null / single-element input?  
[ ] Any off-by-one in loop bounds or window edges?  
[ ] Can any int overflow? (sum, product, mid)  
[ ] Did I dry-run my own example and get the right answer?  
[ ] Did I state time AND space complexity?  
[ ] Did I mention at least one tradeoff or follow-up?
```

If all six are yes, *then* say "I'm confident this is correct — it's $O(\dots)$ time, $O(\dots)$ space."

The meta-lesson

Interviewers are asking one question the whole time: **"Do I want this person on my team when production breaks at 2am?"** Every item above is really about that — do you plan, communicate, check your work, and stay collaborative under pressure? Get those right and a slightly slower solve still passes. Get them wrong and a fast correct solve can still fail.

5. Four Mock Interview Transcripts

The one skill this trains. Everything in this guide comes together here. These are minute-by-minute transcripts of what a **strong candidate actually says out loud** — the words, the questions, the narration, the recovery from a bug, the response to a curveball. Read each one **out loud**. The goal is to make this speech pattern automatic, because under stress you default to silence, and silence loses offers.

[I] = interviewer, [C] = candidate. Timestamps map to the §3 playbook.

Transcript 1 — "Two Sum, then a twist" (Hashing → curveball)

Illustrates: clarifying, brute-force-first, a mid-problem curveball, graceful adaptation.

[I] Given an array of integers and a target, return the indices of two numbers that add up to the target.

[0:00 UNDERSTAND] [C] Let me restate: I'm given an int array and a target int, and I return the two indices whose values sum to the target. A few questions: is the array sorted? Are there exactly one solution, or could there be many? Can I use the same element twice? And how big can the array get?

[I] Not sorted. Exactly one solution. Can't reuse an element. Up to 10^5 elements.

[C] Good — 10^5 means an $O(n^2)$ double loop is $\sim 10^{10}$ operations, too slow, so I want $O(n)$. Let me write an example: `[2, 7, 11, 15]`, target `9` → output `[0, 1]` because `2 + 7 = 9`. Edge case: the two numbers could be equal, like `[3, 3]`, target `6` → `[0, 1]`.

[0:03 PLAN] [C] Brute force is check every pair — $O(n^2)$. To do better: as I scan, for each number `x` I ask "have I already seen `target - x`?" A hash map from value → index answers that in $O(1)$. One pass, $O(n)$ time, $O(n)$ space. Sound good?

[I] Go for it.

[0:06 CODE] [C] I'll narrate as I write.

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>(); // value -> index
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i]; // the complement I'm looking for
        if (seen.containsKey(need)) {
            return new int[] { seen.get(need), i };
        }
        seen.put(nums[i], i); // record after checking, so I don't reus
    }
    return new int[] { -1, -1 };
}
```

I put into the map *after* checking, so I never pair an element with itself.

[0:10 CURVEBALL] [I] Now suppose the array *is* sorted, and I ask you to do it in $O(1)$ extra space.

[C] That changes the tool. Sorted plus $O(1)$ space points me at **two pointers** — one at each end, converge based on the sum. Bigger than target, move the right pointer in; smaller, move the left in.

```
public int[] twoSumSorted(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int sum = nums[lo] + nums[hi];
        if (sum == target) return new int[] { lo, hi };
        else if (sum < target) lo++;           // need a bigger sum
        else hi--;                             // need a smaller sum
    }
    return new int[] { -1, -1 };
}
```

[0:14 TEST] [C] Trace `[2, 7, 11, 15]`, target 9: `lo=0 hi=3, sum 17 > 9 → hi=2; sum 13 > 9 → hi=1; sum 9 == 9 → return [0, 1]`. Correct. Edge: two elements `[1, 2]` target 3 → sum 3, returns immediately. Empty or single element → loop doesn't run, returns `[-1,-1]`, safe.

[0:16 WRAP] [C] The sorted version is $O(n)$ time, $O(1)$ space — better space than the hash map, at the cost of requiring sorted input. If it weren't sorted, sorting first would be $O(n \log n)$ and would lose the original indices, so I'd keep the hash map for the unsorted case.

Why this is strong: de-storied, asked about size and used it, brute-force-first, adapted instantly to the curveball by *re-deriving the pattern from the new constraint* (sorted + $O(1)$ → two pointers), and closed with the tradeoff.

Transcript 2 — "Number of Islands" (Grid DFS, narrating a graph solve)

Illustrates: recognizing a grid as a graph, drawing before coding, the visited decision.

[I] Given a 2D grid of `'1'` (land) and `'0'` (water), count the number of islands. Land connects horizontally and vertically.

[0:00 UNDERSTAND] [C] So islands are connected components of `'1'` s, 4-directionally. Questions: can I modify the grid in place? Are the borders all water or could land touch the edge? Grid dimensions?

[I] You can modify it. Land can touch the edge. Up to 300×300 .

[C] Example:

```
1 1 0
0 1 0
0 0 1
```

That's 2 islands — the connected block top-left, and the single `1` bottom-right.

[0:03 PLAN] [C] A grid is secretly a graph: each cell is a node, its up/down/left/right neighbors are edges. I'll scan every cell; when I hit an unvisited `'1'`, that's a new island — I increment the count and **flood-fill** the whole connected blob so I don't count it again. I'll mark visited by sinking land to `'0'` since I'm allowed to mutate. DFS is simplest. Complexity is $O(\text{rows} \times \text{cols})$ — each cell visited once. Here's the picture:


```
scan row by row; first unvisited 1 starts island #1:
[1] 1  0      flood-fills the connected 1s, sinking them to 0
    0  1  0      when the scan later reaches the lone 1, that's island #2
    0  0 [1]
```

Sound reasonable?

[I] Yes.

[0:06 CODE]

```
public int numIslands(char[][] grid) {
    int rows = grid.length, cols = grid[0].length, count = 0;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '1') {
                count++;
                sink(grid, r, c);          // flood the whole island
            }
        }
    }
    return count;
}

private void sink(char[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length || grid[r][c] != '1') {
        return;                          // out of bounds or not land -> stop
    }
    grid[r][c] = '0';                    // mark visited
    sink(grid, r + 1, c);
    sink(grid, r - 1, c);
    sink(grid, r, c + 1);
    sink(grid, r, c - 1);
}
```

[0:14 TEST] [C] Trace my example: scan hits (0,0)='1' → count=1, sink fills (0,0),(0,1),(1,1). Scan continues, all those are now '0'. Reaches (2,2)='1' → count=2, sink fills it. Result 2. Correct. Edge: all water → count 0. One giant island → count 1, one big flood. 300×300 is 90k cells — recursion depth could be ~90k worst case (a snake shape), which risks a stack overflow.

[0:17 self-caught refinement] [C] Actually, let me flag that: for a pathological grid the DFS recursion could go 90,000 deep and overflow the stack. In production I'd switch to an explicit BFS queue to bound the memory. Want me to write the BFS version?

[I] No, the note is enough.

[0:19 WRAP] [C] $O(\text{rows} \times \text{cols})$ time, and $O(\text{rows} \times \text{cols})$ space worst case for the recursion (or the BFS queue). If I couldn't mutate the grid, I'd use a separate `visited` boolean matrix, same complexity.

Why this is strong: named the grid-as-graph insight, drew the flood-fill before coding, justified the visited strategy, and **proactively caught the stack-depth risk** — a real production concern that signals seniority.

Transcript 3 — "Course Schedule" (Topological sort, recovering from a wrong start)

Illustrates: an initial wrong instinct, catching it, and a clean recovery.

[I] There are `numCourses` courses, some with prerequisites given as pairs `[a, b]` meaning you must take `b` before `a`. Return true if you can finish all courses.

[0:00 UNDERSTAND] [C] So `[a, b]` is a directed edge `b → a`. I can finish everything iff there's no cycle in the dependency graph. Questions: can there be duplicate pairs? Self-loops like `[1, 1]`? How many courses and prerequisites?

[I] Assume no duplicates, no self-loops. Up to 2000 courses.

[C] Example: `numCourses=2, [[1,0]] → true` (take 0, then 1). `[[1,0], [0,1]] → false`, that's a cycle.

[0:03 PLAN — a wrong start, then recovery] [C] My first instinct is to DFS from each node and see if I revisit a node... hmm, wait — let me be careful. Just "have I visited this node before" isn't enough, because a node can be legitimately reachable by two different paths without a cycle. What I actually need is to detect a node that's currently **on my recursion stack** — a back edge. So I'll use three-color DFS: white (unvisited), gray (in progress, on the stack), black (fully done). If I reach a gray node, that's a cycle.

Actually, the cleaner approach for "can finish all" is **Kahn's algorithm** — topological sort by in-degree. Repeatedly take courses with no remaining prerequisites; if I can take all of them, no cycle. Let me go with Kahn's; it's less error-prone than color bookkeeping.

[I] Sounds good.

[0:07 CODE]

```
public boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
    int[] indegree = new int[numCourses];
    for (int[] p : prerequisites) {
        adj.get(p[1]).add(p[0]);           // b → a
        indegree[p[0]]++;
    }
    Queue<Integer> ready = new LinkedList<>();
    for (int c = 0; c < numCourses; c++) if (indegree[c] == 0) ready.add(c);

    int taken = 0;
    while (!ready.isEmpty()) {
        int course = ready.poll();
        taken++;
        for (int next : adj.get(course)) {
            if (--indegree[next] == 0) ready.add(next);
        }
    }
    return taken == numCourses;           // took all → no cycle
}
```

[0:14 TEST] [C] Trace `[[1,0], [0,1]]` : `indegree[0]=1, indegree[1]=1`. Nothing starts at 0 → queue empty → `taken=0 ≠ 2` → false. Correct, it caught the cycle. Trace `[[1,0]]` : `indegree[0]=0` queued; take 0,

decrement 1 to 0, queue it; take 1; taken=2 → true. Correct.

[0:17 WRAP] [C] $O(V + E)$ time and space — build the graph once, each node and edge processed once. The alternative three-color DFS is the same complexity; I chose Kahn's because the in-degree bookkeeping is harder to get wrong than tracking recursion-stack state.

Why this is strong: the candidate *started down a shaky path, caught the flaw in their own reasoning out loud, and corrected to a cleaner algorithm* — with justification. Interviewers love this far more than a memorized right answer, because it shows how you self-correct.

Transcript 4 — "Kth Largest in a stream" (Heap, catching a real bug in the dry-run)

Illustrates: choosing the right data structure, and finding + fixing a bug during testing.

[I] Design a class that, given a stream of integers, can return the `k`-th largest element seen so far at any point. `add(val)` returns the k-th largest after inserting.

[0:00 UNDERSTAND] [C] So it's stateful — I keep adding numbers and each `add` returns the current k-th largest. Questions: is `k` fixed at construction? Can values repeat, and do duplicates count as separate elements? Roughly how many `add` calls?

[I] `k` is fixed up front. Duplicates count separately. Could be millions of adds.

[C] Example, `k=2`: add 5 → only one element, but the problem guarantees we've seen $\geq k$, so say we seed with `[4,5,8,2]`. 2nd largest is 5. `add(3)` → still 5. `add(10)` → now 2nd largest is 8.

[0:03 PLAN] [C] I want the k-th largest fast on every add. If I sorted every time that's $O(n \log n)$ per add — too slow for millions. The trick: keep a **min-heap of size k** holding the k largest elements seen. The heap's *minimum* (its root) is exactly the k-th largest. On add, push; if the heap exceeds size k, pop the smallest. Peek is $O(1)$, add is $O(\log k)$. Picture:

```
min-heap of the k=2 largest, root = k-th largest:
  after [4,5,8,2] kept largest 2 -> {5, 8}, root 5 = 2nd largest
  add 10 -> {8, 10} (5 popped), root 8
```

[I] Good.

[0:06 CODE]

```

class KthLargest {
    private final PriorityQueue<Integer> minHeap; // holds the k largest so far
    private final int k;

    public KthLargest(int k, int[] nums) {
        this.k = k;
        this.minHeap = new PriorityQueue<>(); // natural order = min-heap
        for (int n : nums) add(n);
    }

    public int add(int val) {
        minHeap.offer(val);
        if (minHeap.size() > k) minHeap.poll(); // drop the smallest
        return minHeap.peek();
    }
}

```

[0:12 TEST — catches a bug] [C] Let me dry-run $k=2$, seed `[4,5,8,2]`. `add(4)`: heap `{4}`. `add(5)`: `{4,5}`. `add(8)`: `{4,5,8}` size $3 > 2 \rightarrow$ poll min 4 $\rightarrow$ `{5,8}`. `add(2)`: `{2,5,8}` $\rightarrow$ poll 2 $\rightarrow$ `{5,8}`. Good, root is 5 = 2nd largest. Now the live call `add(10)`: `{5,8,10}` $\rightarrow$ poll 5 $\rightarrow$ `{8,10}`, peek 8. Correct.

Wait — one problem. In the constructor I call `add` before the heap has k elements. On the very first `add(4)`, `peek()` returns 4, but there's no valid "k-th largest" yet. If the grader calls `add` during seeding and reads the return, that's misleading. Let me check the contract... the problem guarantees the seed has at least $k-1$ elements and every *live* `add` brings it to $\geq k$, so `peek()` is only meaningfully read when $\text{size} \geq k$. My code is actually correct for the stated contract — but I'm glad I checked, because if seeding could be shorter than $k-1$, `peek()` on a heap of size $< k$ would return the wrong value. I'd guard with `return minHeap.size() < k ? -1 : minHeap.peek();` to be safe.

[0:17 WRAP] [C] Construction is $O(n \log k)$, each `add` is $O(\log k)$, space $O(k)$. A min-heap of size k beats sorting ($O(n \log n)$ per query) and beats a max-heap of everything ($O(n)$ space, $O(k)$ per query). For millions of adds, $O(\log k)$ per add is the win.

Why this is strong: picked the non-obvious right structure (min-heap for k-th *largest*) and explained *why* the root is the answer, then **used the dry-run to surface an edge in the contract** and stated the defensive guard — rigor without over-engineering.

What all four have in common

Read them again and notice the shared spine — it's the §3 playbook every time:

1. **Restate + clarify with questions** (especially about size $\rightarrow$ complexity).
2. **Write an example, including an edge case.**
3. **Brute force / first instinct out loud, then the optimization with its pattern named.**
4. **Get a nod before coding.**
5. **Narrate while coding**, clean Java.
6. **Dry-run the example by hand** — this is where bugs and edge cases surface.
7. **Close with complexity + a tradeoff or follow-up.**

The technical content in these four is straight out of Vol 1. What makes them *interview wins* is the **process and the talking**. That's the whole point of this volume: you already know the algorithms — now

rehearse performing them out loud until this spine is muscle memory.

Closing note

Knowledge is necessary but no longer your bottleneck. Do **timed, spoken, hand-tested** mock solves using the §3 playbook and the §4 checklist, and read these transcripts aloud until the narration feels natural. Ten problems performed this way will lift your pass rate more than another fifty problems solved silently on paper.

Good luck — you're well-prepared. Go perform it.